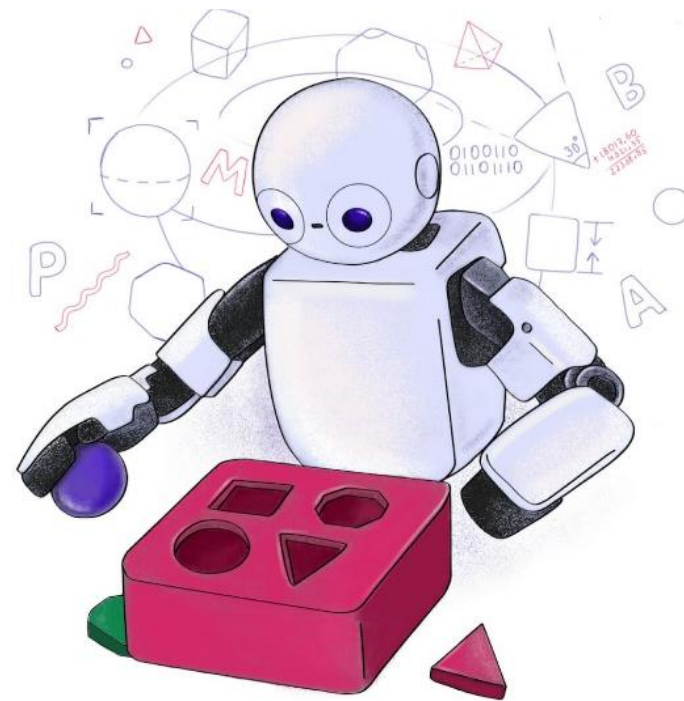


TP558 - Tópicos avançados em Machine Learning:

Deep Deterministic Policy Gradient (DDPG)



Abertura

REINFORCEMENT LEARNING

Agente aprende interagindo — observa s , executa a , recebe r

Q-LEARNING

Valor de cada par (s, a) guardado numa tabela

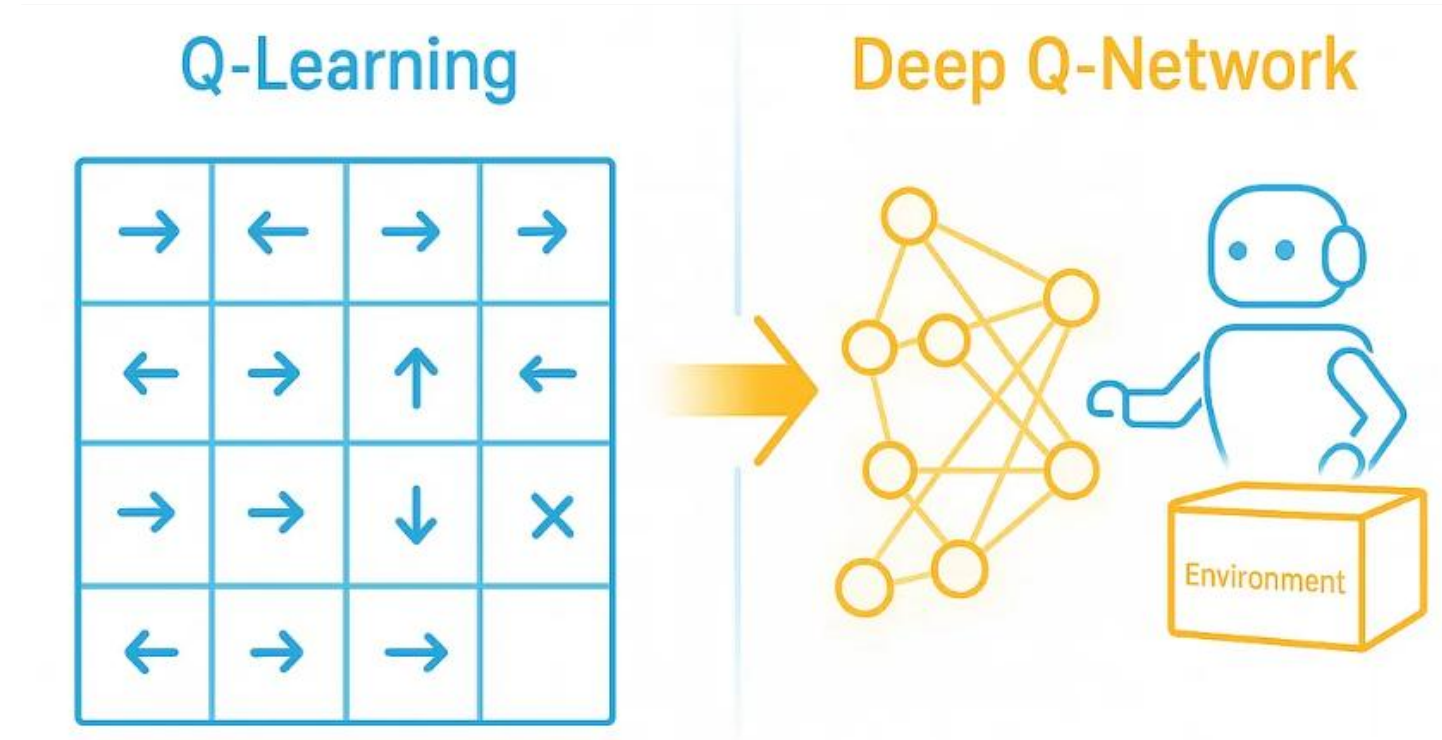
$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

Limite — só existe se as ações forem poucas e discretas

DEEP Q-NETWORK (DQN)

Rede neural no lugar da tabela — $Q(s, a | \theta)$

Mas — o operador $\max_{a'}$ continua exigindo ações discretas



Por que não basta discretizar?

Braço robótico com 7 graus de liberdade

3 VALORES / JUNTA

2.187

ações discretas

10 VALORES / JUNTA

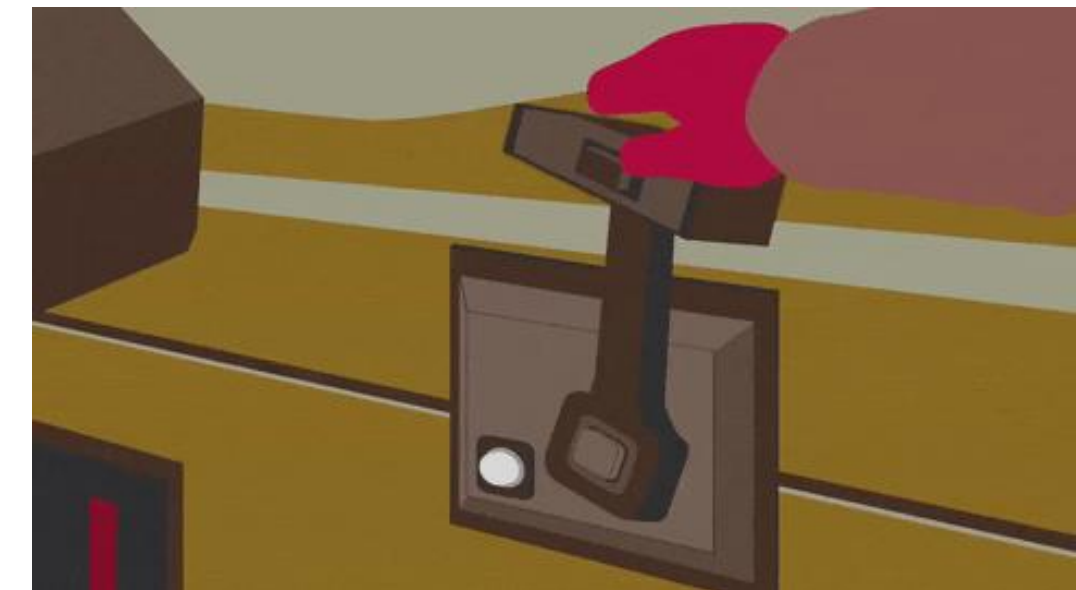
10.000.000

de ações

Crescimento exponencial — com os graus de liberdade

Estrutura descartada — perde-se a noção de que ações vizinhas têm valores parecidos

É preciso um método que opere nativamente no contínuo



DDPG — Deep Deterministic Policy Gradient

DOMÍNIO

Ações contínuas

ORIGEM

DPG + ideias do DQN

ARQUITETURA

Actor-Critic — uma rede decide, outra avalia

ESTABILIDADE

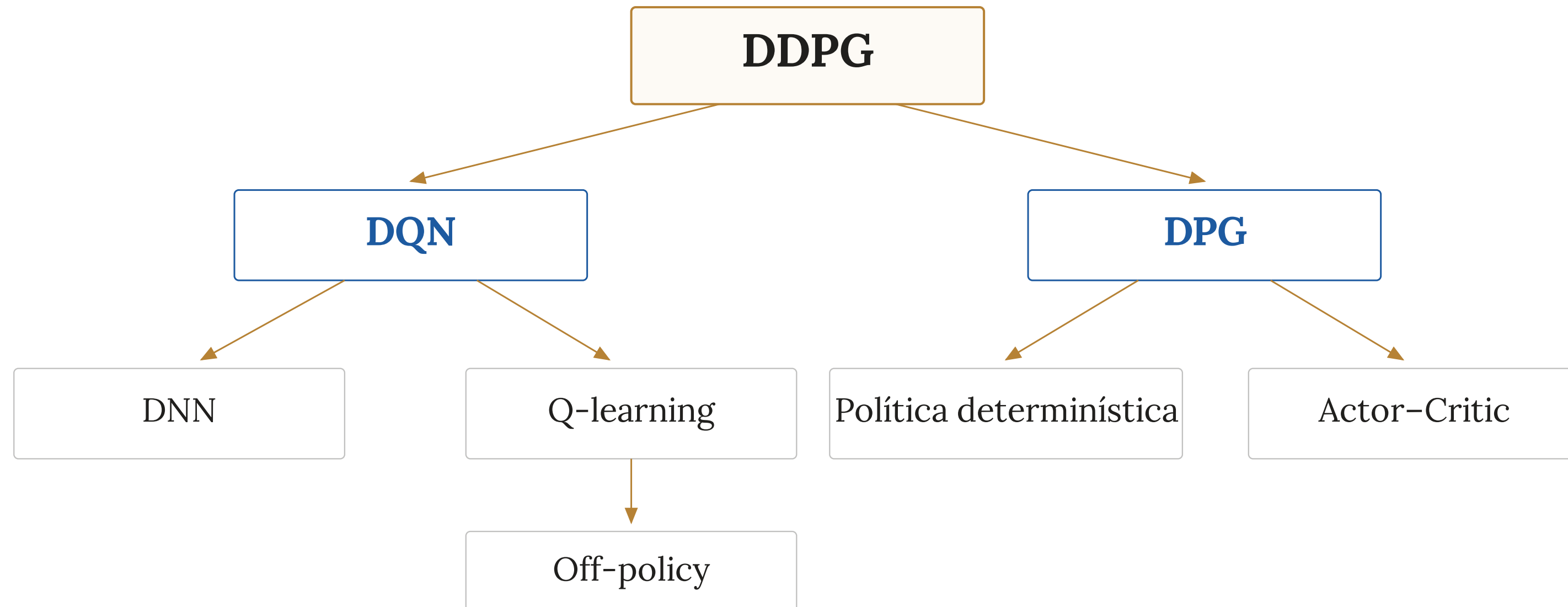
Replay Buffer e Target Networks

NATUREZA

Model-free, off-policy e end-to-end



Fundamentação teórica



Policy Gradient

POLÍTICA PARAMETRIZADA

Otimizar θ por ascensão de gradiente sobre o retorno esperado

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \gamma^t r_t \right]$$

TEOREMA DO POLICY GRADIENT

Gradiente em função de $\log \pi$ e do valor Q

$$\nabla_{\theta} J = \mathbb{E} \left[\nabla_{\theta} \log \pi_{\theta}(a|s) Q^{\pi}(s, a) \right]$$

Problema — estimar Q^{π} por Monte Carlo (REINFORCE): alta variância



Solução — aproximar Q^{π} com uma segunda rede treinada



Nasce o Critic — Actor-Critic = política parametrizada + valor aprendido

O DDPG aplica a lógica ao caso determinístico: $\mu(s)$ em vez de $\pi(a|s)$ — daí o Teorema DPG

Actor: definição e arquitetura

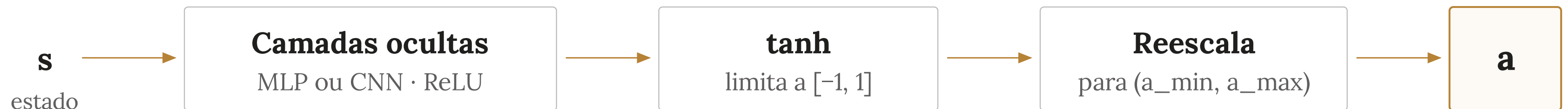
POLÍTICA DETERMINÍSTICA

$$a = \mu(s | \theta^\mu)$$

Uma ação, sem amostragem — diferente de $\pi(a|s)$, que devolve uma distribuição

IDEIA-CHAVE

“O que devo fazer neste estado?”



A ação resultante é sempre fisicamente válida

Objetivo do Actor e Teorema DPG

FUNÇÃO OBJETIVO

Maximizar o valor das ações que produz, segundo o Critic

$$J(\theta^\mu) = \mathbb{E}_{s \sim \rho^\beta} [Q(s, \mu(s|\theta^\mu) | \theta^Q)]$$

ρ^β — estados sob a política de comportamento $\beta = \mu + \text{ruído}$ (off-policy)

TEOREMA DPG

Gradiente determinístico — regra da cadeia em duas partes

$$\nabla_{\theta^\mu} J \approx \mathbb{E} [\nabla_a Q(s, a|\theta^Q) \big|_{a=\mu(s)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)]$$

$$\nabla_a Q$$

Vem do Critic — quanto o valor muda se eu mudar a ação

$$\nabla_{\theta^\mu} \mu$$

Vem do Actor — quanto a ação muda se eu mudar os pesos

O produto dá a direção em que mover θ^μ para que a ação tenha Q maior

Minibatch, atualização e diferenciabilidade

ESTIMATIVA POR MINIBATCH

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a | \theta^Q) \Big|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s | \theta^\mu) \Big|_{s_i}$$

ASCENSÃO DE GRADIENTE

$$\theta^\mu \leftarrow \theta^\mu + \alpha_{actor} \nabla_{\theta^\mu} J$$

A ACTOR
 10^{-4}

A CRITIC
 10^{-3}

CONDIÇÃO DE DIFERENCIABILIDADE

Só funciona porque $Q(s,a)$ é diferenciável em a

É o que restringe o DDPG ao contínuo — em espaço discreto não existe $\nabla_a Q$

CONSEQUÊNCIA

Dispensa importance sampling

Não há integral sobre distribuição de ações — o off-policy é natural

Exploração: ruído de Ornstein–Uhlenbeck

O PROBLEMA

Uma política determinística não explora sozinha

Mesmo estado, sempre a mesma ação

A SOLUÇÃO

Ruído aditivo na ação — apenas durante o treino

$$a_t = \mu(s_t | \theta^\mu) + \mathcal{N}_t$$

NO PAPER ORIGINAL

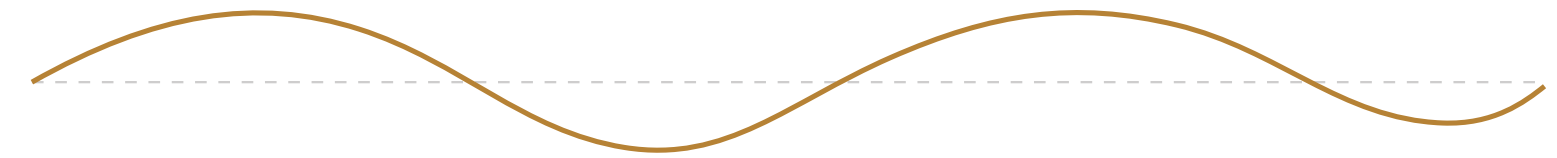
$\mathcal{N}$ é um processo de Ornstein–Uhlenbeck

$$dx_t = \theta(\bar{\mu} - x_t) dt + \sigma dW_t$$

Gaussiano — independente a cada passo



Ornstein–Uhlenbeck — correlacionado no tempo



POR QUÊ OU E NÃO GAUSSIANO

Gera ruído correlacionado no tempo — adequado a sistemas físicos com inércia, onde perturbações independentes se cancelam

Em teste/execução — $a_t = \mu(s_t | \theta^\mu)$, sem ruído: puramente determinístico

Critic: definição e valor-alvo

FUNÇÃO VALOR-AÇÃO

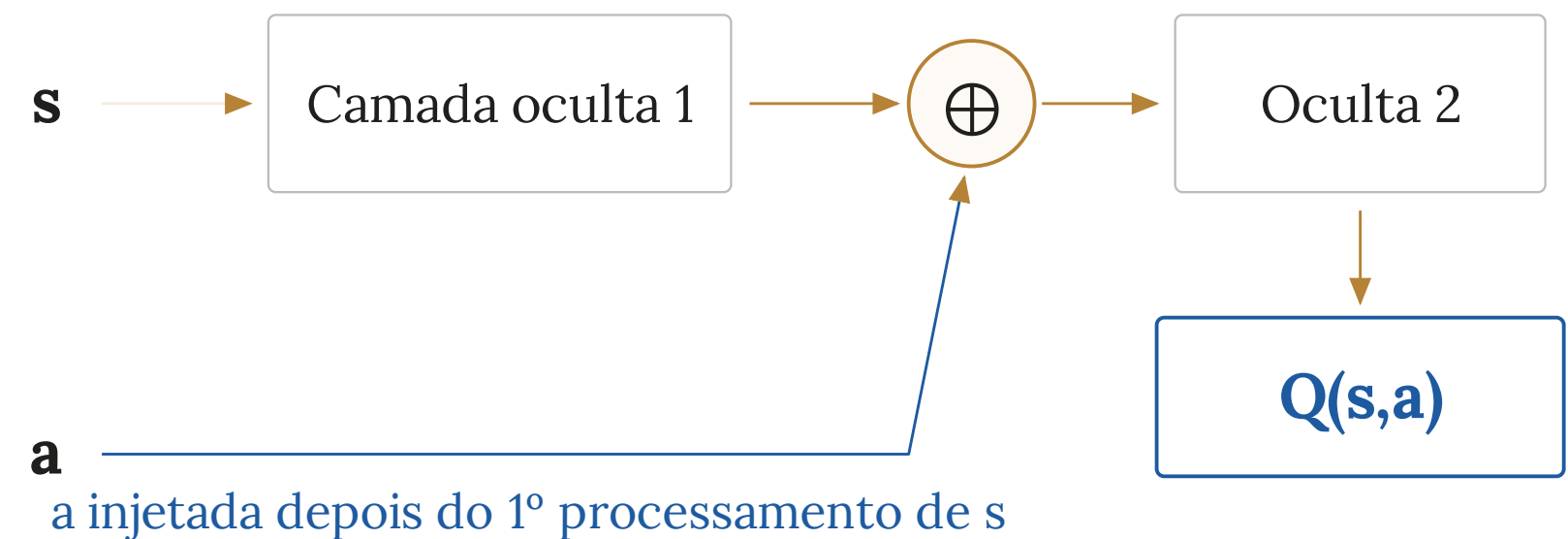
$Q(s, a \mid \theta^Q)$ — estima o retorno esperado a partir dali
Avalia; não escolhe ações

“Quão boa é esta ação neste estado?”

VALOR-ALVO (BELLMAN)

$$y = r + \gamma(1 - d) Q'(s', \mu'(s' \mid \theta^{\mu'}) \mid \theta^{Q'})$$

$d = 1$ se o episódio terminou $\rightarrow y = r$, sem bootstrap



DQN $\max_a Q(s', a')$



DDPG $\mu'(s')$ — o máximo é intratável no contínuo

Critic: perda MSBE e atualização

REGRESSÃO SUPERVISIONADA

Minimizar o erro de Bellman quadrático médio

$$L(\theta^Q) = \mathbb{E}_{\mathcal{D}} [(Q(s, a|\theta^Q) - y)^2]$$

VERSÃO MINIBATCH – O QUE O CÓDIGO FAZ

$$L(\theta^Q) = \frac{1}{N} \sum_i (Q(s_i, a_i|\theta^Q) - y_i)^2$$

DESCIDA DE GRADIENTE

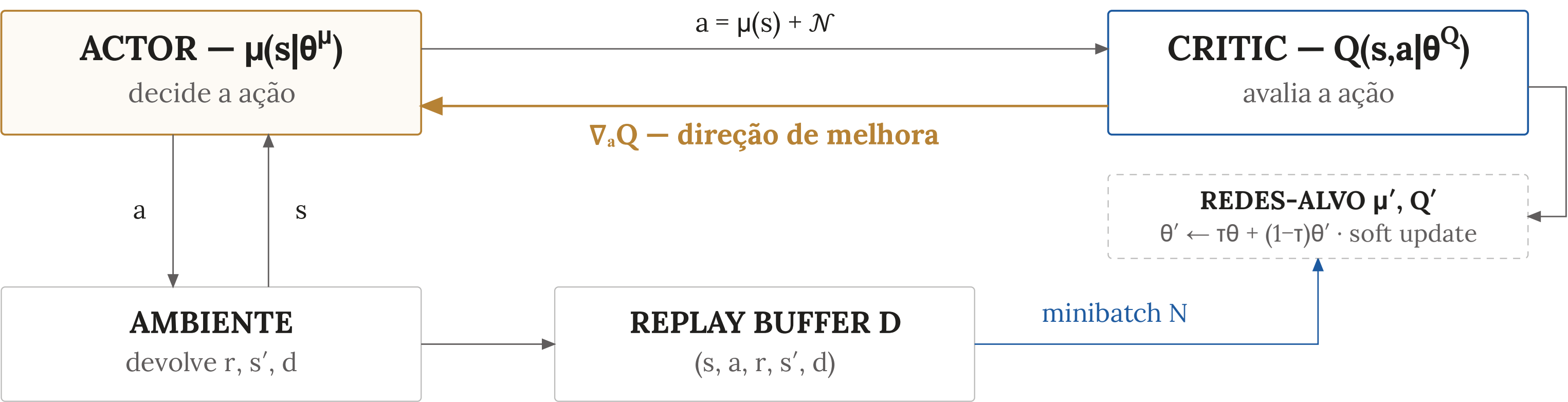
$$\theta^Q \leftarrow \theta^Q - \alpha_{critic} \nabla_{\theta^Q} L(\theta^Q)$$

REDE-ALVO – POLYAK AVERAGING

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

Sem a rede-alvo, y mudaria a cada passo – o Critic perseguiria um alvo móvel

O ciclo Actor ↔ Critic



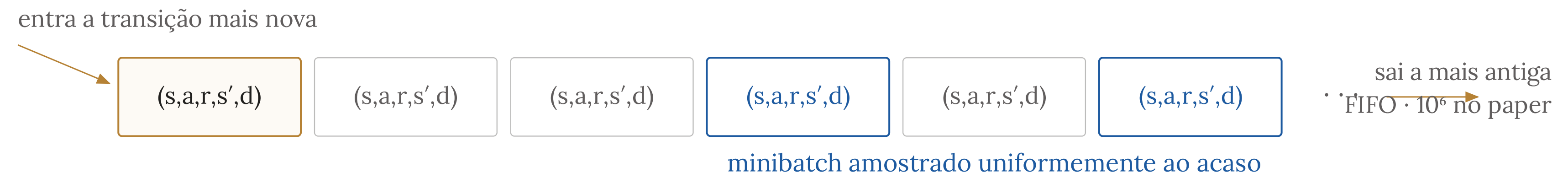
$y = r + \gamma(1-d) \cdot Q'(s', \mu'(s'))$ — o alvo do Critic vem das redes-alvo

Treino do Actor — θ^Q fixo; o gradiente apenas atravessa o Critic

Treino do Critic — θ^μ fixo; o Actor-alvo só fornece a'

Em produção — quem atua é sempre o Actor

Replay Buffer



POR QUÊ

O SGD assume amostras i.i.d. — mas transições consecutivas são fortemente correlacionadas no tempo

EFEITO

Amostrar ao acaso quebra a correlação e aproxima a condição i.i.d.

EFICIÊNCIA DE AMOSTRA

Cada transição é reutilizada em vários updates — não descartada após um uso

CARÁTER OFF-POLICY

As transições vêm de políticas passadas — aprender com elas só é possível off-policy

Extensão posterior: Prioritized Experience Replay — amostra por erro TD

Target Networks: o problema do alvo móvel

O ALVO DEPENDE DOS PRÓPRIOS PESOS

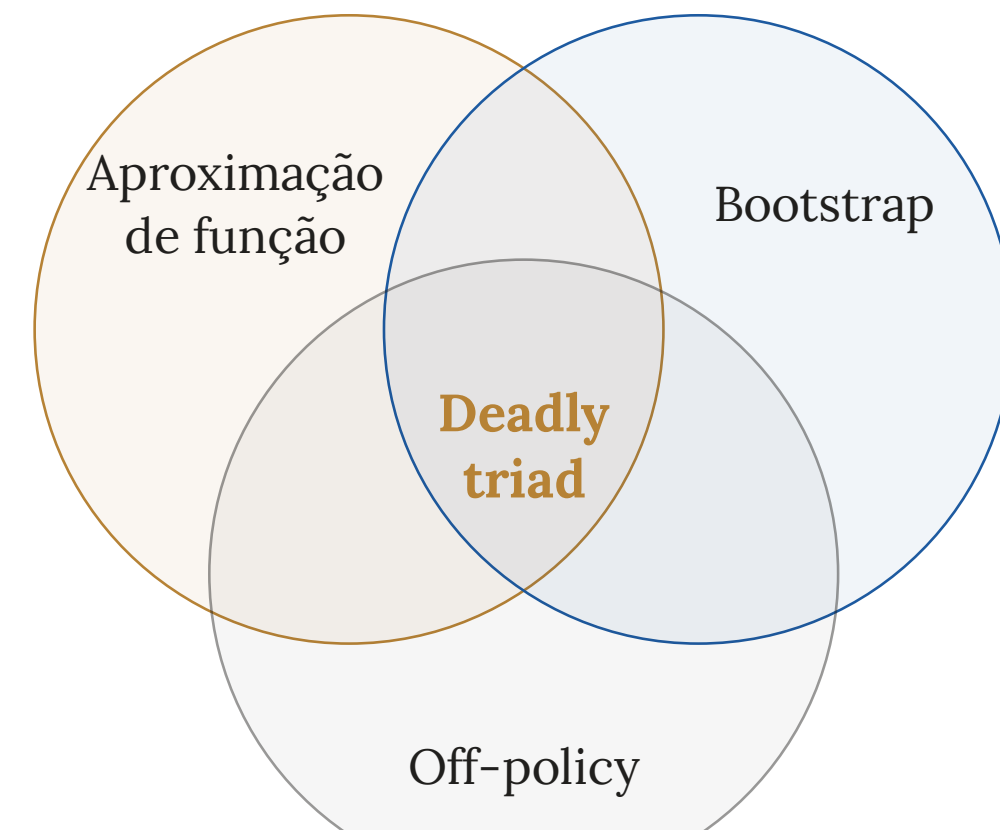
$$y = r + \gamma Q(s', a' | \theta)$$

Mesma rede prevê e define o alvo — cada passo de gradiente move o alvo: oscilação e, no limite, divergência

SOLUÇÃO

Uma cópia defasada dos parâmetros, usada apenas para calcular y

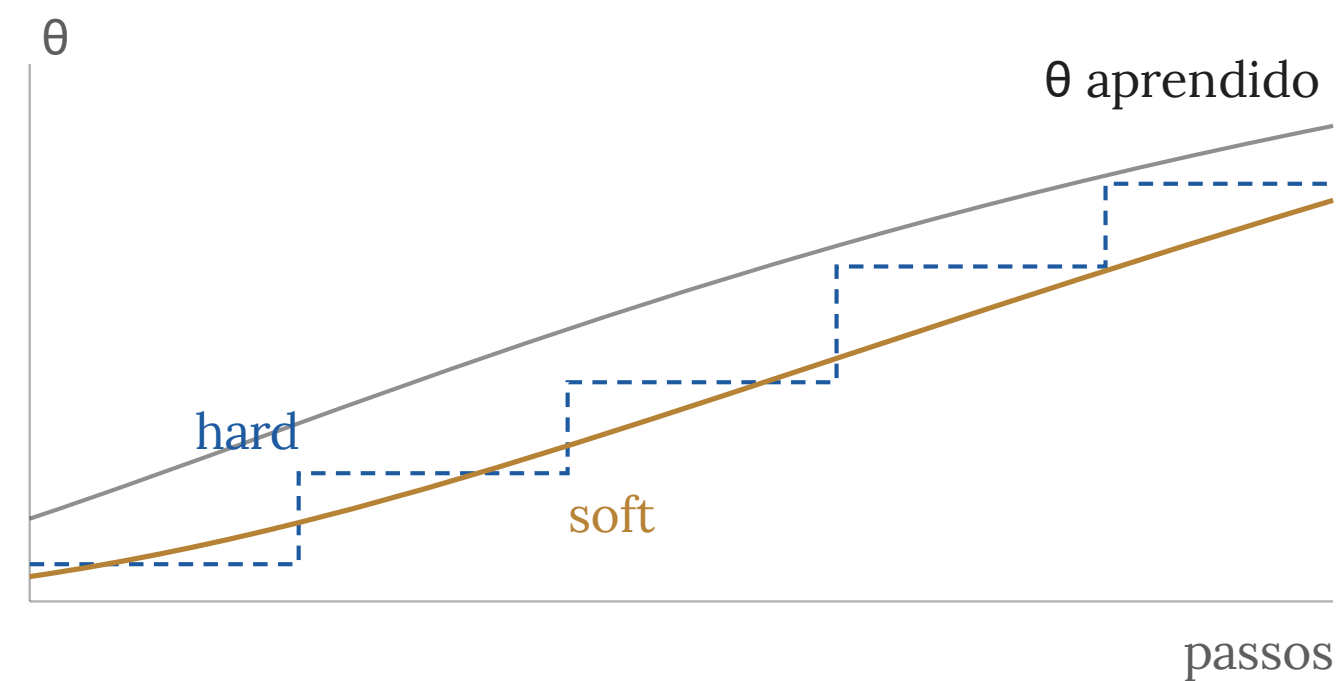
Rigor — as redes-alvo não eliminam a triad: tornam o treino empiricamente estável apesar dela. Não há garantia de convergência



O DDPG tem os três simultaneamente: rede neural + alvo bootstrapado + replay buffer (Sutton & Barto)

Hard update (DQN) vs soft update (DDPG)

	DQN	DDPG
Atualização	Hard — cópia periódica	Soft — Polyak, contínua
Regra	$\theta' \leftarrow \theta$, a cada C passos	$\theta' \leftarrow \tau\theta + (1-\tau)\theta'$, a cada passo
Espaço de ação	Discreto	Contínuo
O alvo sustenta	$\max_a Q$ — um valor	$\nabla_a Q$ — um gradiente
Salto abrupto	Tolerável	Crítico



Por que soft — o Actor depende de $\nabla_a Q$, não só do valor: um salto abrupto no alvo desestabiliza o gradiente

Trade-off de τ — muito pequeno: alvo estável, aprendizado lento; muito grande: o desacoplamento deixa de existir

Algoritmo do pêndulo invertido

REDE NEURAL DO ATOR

```
def get_actor():
    # Initialize weights between -3e-3 and 3-e3
    last_init = keras.initializers.RandomUniform(minval=-0.003, maxval=0.003)

    inputs = layers.Input(shape=(num_states,))
    out = layers.Dense(256, activation="relu")(inputs)
    out = layers.Dense(256, activation="relu")(out)
    outputs = layers.Dense(1, activation="tanh", kernel_initializer=last_init)(out)

    # Our upper bound is 2.0 for Pendulum.
    outputs = outputs * upper_bound
    model = keras.Model(inputs, outputs)
    return model
```

ENTRADA Estado do ambiente

SAIDAS

Ação contínua — Camada Dense de 1 neurônio com ativação tanh, escalada pelo torque máximo (± 2.0).

ACTIVAÇÕES

Funções relu para camadas ocultas e tanh para a camada de saída

ACTOR

Recebe o estado e devolve uma ação (política determinística), por isso usa tanh (limita entre -1 e 1) e depois escala pelo upper_bound (o torque máximo do pêndulo).

Arquitetura

REDE NEURAL DO CRÍTICO

```
def get_critic():
    # State as input
    state_input = layers.Input(shape=(num_states,))
    state_out = layers.Dense(16, activation="relu")(state_input)
    state_out = layers.Dense(32, activation="relu")(state_out)

    # Action as input
    action_input = layers.Input(shape=(num_actions,))
    action_out = layers.Dense(32, activation="relu")(action_input)

    # Both are passed through separate layer before concatenating
    concat = layers.Concatenate()([state_out, action_out])

    out = layers.Dense(256, activation="relu")(concat)
    out = layers.Dense(256, activation="relu")(out)
    outputs = layers.Dense(1)(out)

    # Outputs single value for give state-action
    model = keras.Model([state_input, action_input], outputs)

    return model
```

ENTRADA Recebe o **Estado** e a **Ação** separadamente

SAIDAS

Valor escalar $Q(s, a)$ — que avalia a qualidade da ação dada o estado

ACTIVAÇÕES

Funções relu para todas as camadas ocultas

CRITIC

Recebe estado e ação, são concatenados, e devolve um único número $Q(s,a)$, "quão boa é essa ação nesse estado"

Treinamento

```
with tf.GradientTape() as tape:
    target_actions = target_actor(next_state_batch, training=True)
    y = reward_batch + gamma * target_critic([next_state_batch, target_actions], training=True)
    critic_value = critic_model([state_batch, action_batch], training=True)
    critic_loss = tf.reduce_mean(tf.square(y - critic_value))
```

```
with tf.GradientTape() as tape:
    actions = actor_model(state_batch, training=True)
    critic_value = critic_model([state_batch, actions], training=True)
    actor_loss = -tf.reduce_mean(critic_value)
```

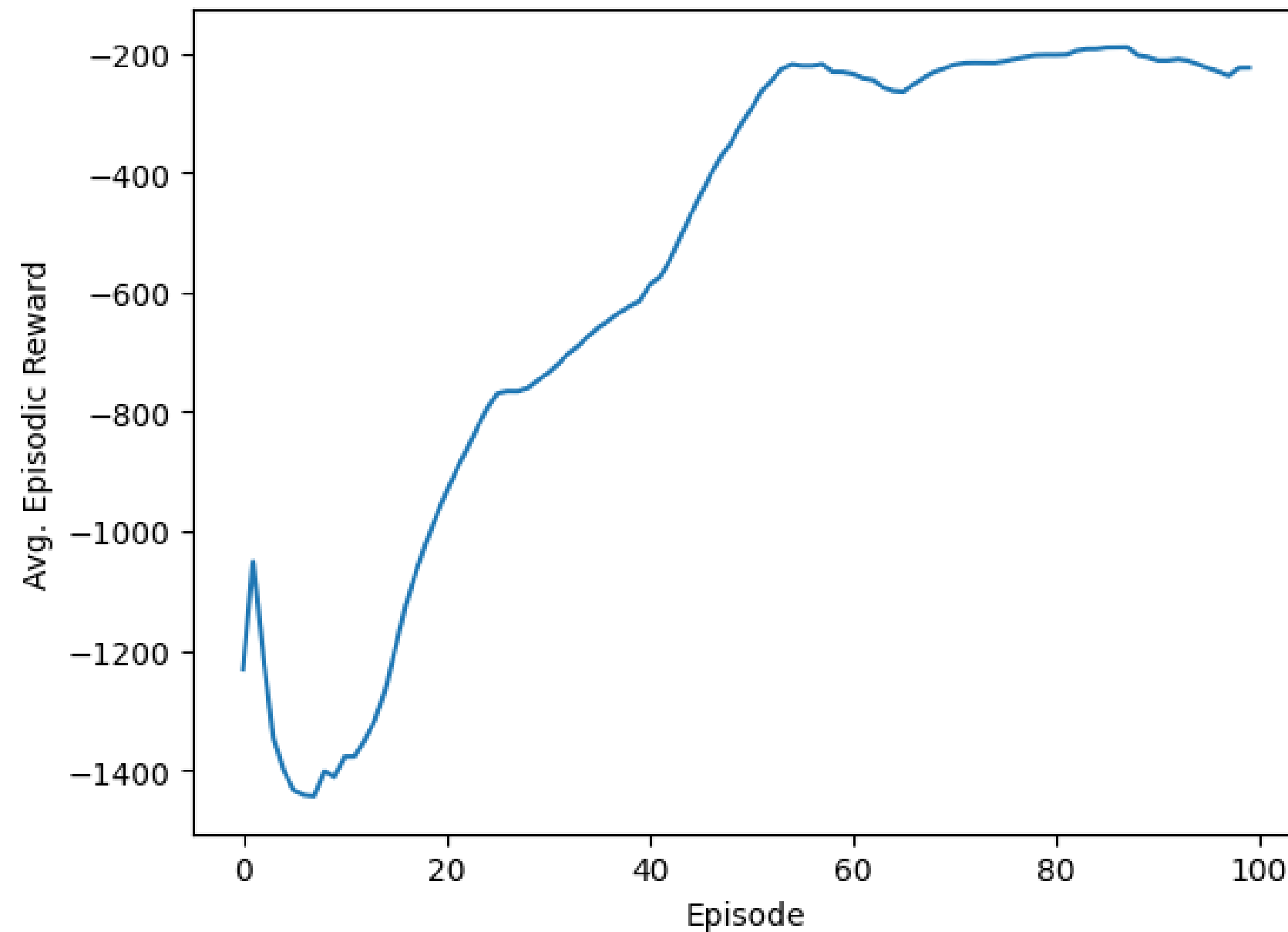
```
update_target(target_actor, actor_model, tau)
update_target(target_critic, critic_model, tau)
```

Critic aprende por erro de diferença temporal (equação de Bellman): o alvo y usa as redes-alvo, que são atualizadas devagar ($\tau = 0.005$) para estabilizar o treino.

Actor aprende maximizando a nota dada pelo Critic — por isso o sinal negativo na loss (o otimizador minimiza).

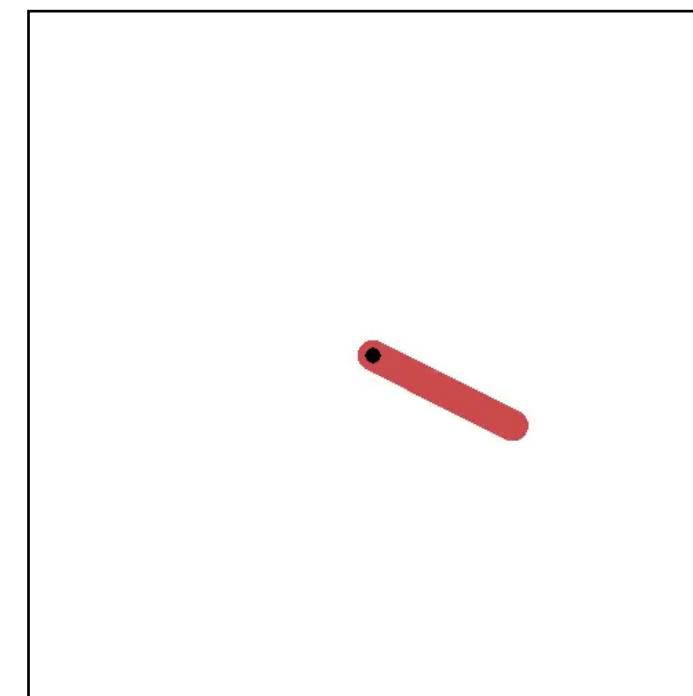
A cada passo: agir com ruído (exploração), guardar no replay buffer, treinar com um minibatch aleatório do buffer, e dar o "soft update" nas redes-alvo.

Curva de Recompensa



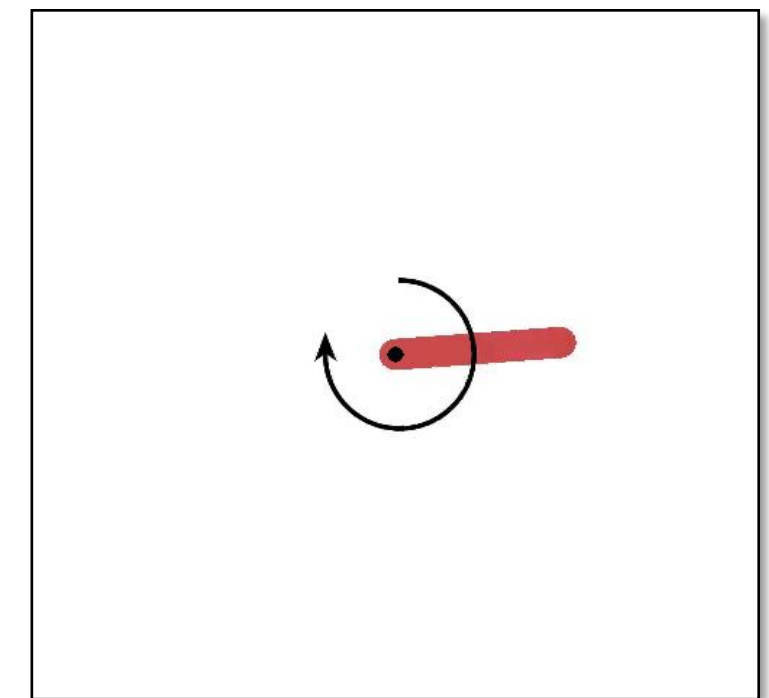
A recompensa média por episódio em função do número de episódios

ANTES DO TREINAMENTO



Antes: o pêndulo cai e balança sem controle, política ainda aleatória.

DEPOIS 100 EPISÓDIOS



Depois: o pêndulo fica equilibrado perto da vertical — confirmação de que o par Actor-Critic aprendeu a tarefa

Vantagens

DOMÍNIO

Ações contínuas de alta dimensionalidade, nativamente

EFICIÊNCIA DE AMOSTRA

Off-policy + replay buffer

VARIÂNCIA

Gradiente determinístico — sem integral sobre distribuição de ações

ATUALIZAÇÃO

Bootstrap (TD) — a cada passo, sem esperar o fim do episódio

ESCOPO

Model-free e end-to-end — funciona a partir de pixels

Limitações

FRAGILIDADE	Sensível a hiperparâmetros e a sementes (Henderson et al., 2018)	
VIÉS DE Q	Sobrestimação — o Critic superestima e o Actor, maximizando Q, é empurrado para as regiões de maior erro	→ TD3
EXPLORAÇÃO	Ingênua — ruído aditivo não se adapta à incerteza; falha com recompensa esparsa	→ SAC
EXPRESSIVIDADE	Política determinística — inadequada quando o ótimo exige aleatoriedade	cenários adversariais
TEORIA	Sem garantia de convergência — deadly triad	

TD3: clipped double-Q, delayed updates, target policy smoothing · SAC: entropia máxima

Referências

- [1] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” in Proc. Int. Conf. Learn. Representations (ICLR), 2016.
 - [2] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic policy gradient algorithms,” in Proc. 31st Int. Conf. Mach. Learn. (ICML), Beijing, China, 2014, pp. 387–395.
-

Link colab :

https://colab.research.google.com/drive/1xeYSWCLI75FY_sKBRdhh-NnWdPAYyW2-?usp=sharing#scrollTo=Ect__fbVj28S

Obrigado!

Inatel

Perguntas?

Quiz

<https://forms.gle/mpxsuGtS9JstNWa77>

Pseudocode**Algorithm 1** Deep Deterministic Policy Gradient

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta$ ,  $\phi_{\text{targ}} \leftarrow \phi$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_{\theta}(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for however many updates do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute targets

```

$$y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$$

```

13:    Update Q-function by one step of gradient descent using

```

$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi}(s, a) - y(r, s', d))^2$$

```

14:    Update policy by one step of gradient ascent using

```

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi}(s, \mu_{\theta}(s))$$

```

15:    Update target networks with

```

$$\begin{aligned} \phi_{\text{targ}} &\leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned}$$

```

16:    end for
17:  end if
18: until convergence

```